

PrimePC



Memoria del proyecto

Universidad : Universidad Complutense de Madrid.

Asignatura : Modelado de Software.

Curso: 3ºE

Profesor: Antonio Navarro

Autores: Álvaro Adrada, Pablo Aguilar, Sergio Barbas, Laura Blanco, Jorge Calvo, Adrián García, Mateo Gutierrez, Rita Maldonado, Robert Nicolae, Adrián Rodríguez, Darío Romero, Sergio Tarancón.

Índice

1. Introducción.....	2
2. Arquitectura.....	2
3. Patrones de diseño.....	2
4. Esquema de los diagramas.....	4
5. Repositorios.....	4
6. Conclusiones.....	4

1. Introducción

En este documento resumimos el trabajo realizado desde septiembre. Su inicio se produjo con la elaboración de la SRS y de la parte DAO, finalizando con la actual entrega extraordinaria de la parte JPA.

Durante estas semanas hemos utilizado IBM-RSAD como herramienta case y GitHub como sistema de control de versiones.

El proyecto está dividido en dos partes, el modelo de diseño, que cuenta con los diagramas de clase, componentes, secuencia y despliegue y la otra parte es el código, programado en java. La aplicación cuenta con una interfaz sencilla y personalizada para poder desarrollar todas las actividades descritas en los diagramas.

2. Arquitectura

Las bases de nuestro proyecto residen en la arquitectura multicapa, dividida en tres, integración, negocio y presentación. Vamos a desarrollar qué es lo que incluye cada una:

Integración: capa encargada de la comunicación entre la base de datos relacional y el código. Los DAOs (data access object) están implementados para realizar dicha función. En esta nueva versión del proyecto hemos incluido al Entity Manager y manejado la persistencia en la base de datos.

Negocio: capa en la que están implementados los servicios de aplicaciones de cada una de las tablas de la base de datos. Se trata de la capa en la que se realizan todas las comprobaciones pertinentes para garantizar el correcto funcionamiento de la aplicación.

Presentación: es la capa con la que el usuario tiene contacto y con la que interactúa. Las vistas se han construido con Java Swing. También se encuentra el controlador que llamará al servicio de aplicaciones pertinente y tras ejecutar una acción específica devolverá una nueva vista con el resultado.

3. Patrones de diseño

En este apartado vamos a mencionar los patrones empleados para dar soluciones simples y elegantes a problemas específicos del diseño de software orientado a objetos.

Patrón Modelo Vista Controlador (MVC):

Sobre este patrón hemos elaborado nuestra práctica. Así las vistas solo pueden acceder al modelo a través del controlador. Se reduce el acoplamiento, volviendo los distintos elementos de la práctica independientes y aumentando la calidad y mantenibilidad de nuestro software.

Patrón DAO

Se utiliza para acceder y manipular los datos en la base de datos. Cada DAO contiene los métodos CRUD (Create, Read, Update, Delete) específicos de cada entidad.

Patrón Transferencia:

Este patrón forma parte de la arquitectura multicapa, funciona implementando *transfers* para favorecer la comunicación entre capas.

Patrón de creación Abstract Factory :

Lo utilizamos en tres ocasiones, para crear los DAOs en integración, los servicios de aplicaciones en negocio y para construir las vistas en presentación.

Patrón de creación Singleton :

Está presente en el controlador y las factorías abstractas, garantizando una única instancia para cada clase.

Patrón Servicio de Aplicación:

El Servicio de Aplicación es el encargado de manejar la concurrencia y gestionar el flujo de trabajo.

Patrón de comportamiento Observador:

Nuestro controlador funciona como un observador ya que es capaz de notificar y actualizar a otros objetos que dependen de él sin hacer suposiciones sobre quiénes son dichos objetos.

Patrón Service to worker

Este patrón es el encargado de llevar a cabo el manejo de peticiones y la invocación de lógica del negocio antes de pasar el control a la vista.

Patrón Domain Store

Este patrón es el encargado de gestionar la concurrencia y transaccionalidad de forma pesimista en la parte DAO y forma optimista en la actual parte de JPA.

Patrón Objeto del negocio

Este patrón se usa en la capa de negocio y estos objetos se encargan de la lectura y escritura de los datos.

Patrón Command y FactoryCommand:

Este patrón se usa para parsear las peticiones según el comando y ejecutar su correspondiente servicio de aplicación.

4. Esquema de los diagramas

Modelo del dominio

Modelo del dominio

Diagramas de despliegue

Estructura de la base de datos DAO y JPA

Diagrama de despliegue

Diagrama de componentes

Modelo del Diseño (mencionamos exclusivamente la parte JPA)

Diagramas de clase

- Integración: Entity Manager
- Negocio: Todos
- Presentación: Todos

Diagramas de secuencia

- Negocio: Transporte (todos), Factura (abrir factura, cerrar factura, insertar paquete a factura, devolución y eliminar paquete de factura), Remitente (alta, función polimórfica), Trabajador (baja y leer), Ruta (vincular ruta y trabajador)
- Presentación: Transporte (todos)

5. Repositorios

Repositorio en el que se encuentra el modelo del diseño:

<https://github.com/anavarro-fdi-ucm-es/ms2526primepcmod.git>

Repositorio en el que se encuentra el código:

<https://github.com/anavarro-fdi-ucm-es/ms2526primepccod.git>

Están incluidos el código y la base de datos.

6. Conclusiones

En este proyecto nos hemos basado en el que elaboramos en IS2, al cuál hemos añadido gestión de transacciones y concurrencia. También están incluidas dos *Querys* y en esta nueva versión contamos con la función polimórfica y la persistencia empleando Jakarta Persistence.